SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 Design and Analysis of Algorithms

ASSIGNMENT

Name:Y Sai Chetan
Regno:192473019

Assignment 1 - CO1

CO1 Coverage: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Course Outcome: CO1 – Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm **5 Marks**

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach

- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**. Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit. **Evidence Expected**

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**
- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case

- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis **3 Marks**

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation **3 Marks**

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation **4 Marks**

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical

technique Discuss:

- Why the selected method is applicable

- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms **4 Marks**

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance **3 Marks**

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified Present the results using appropriate:
- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour **4 Marks**

Compare your theoretical analysis with your experimental results.
Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved
- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.
Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.
However, new student records are added only **once per day**.
Re-evaluate your original design in response to these changed operating conditions.
You may:
RETAIN → MODIFY → or REDESIGN
your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence. Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks Assigned	Marks Obtained
Task 1 – Algorithm Design	5	
Task 2 – Basic Operation	2	
Task 3 – Mathematical Complexity Analysis	6	
Task 4 – Best/Average/Worst-Case Analysis	4	
Task 5 – Asymptotic Analysis	3	
Task 6 – Recurrence Formulation	3	
Task 7 – Recurrence Solution	4	
Task 8 – Implementation & Testing	4	
Task 9 – Experimental Performance	3	
Task 10 – Theory vs Experiment	4	
Task 11 – Final Design Recommendation	4	
Subtotal	**42**	
Task 12 – Day-4 Design Challenge	**8**	
TOTAL	**50**	

DESIGN, ANALYSIS AND VALIDATION OF A SEARCH STRATEGY FOR A STUDENT RECORD SYSTEM

TASK 1: DESIGN THE ALGORITHMS

Three strategies were chosen so that each one rests on a different idea: brute force, decrease-and-conquer, and a space-time trade-off. Every record is identified by a unique 9-digit Registration Number, and **n** denotes the number of records.

Approach A: Linear (sequential) search, iterative

- **Design strategy:** Brute force.
- **Idea:** Start at the first record and move forward one record at a time, stopping as soon as the key matches or the records run out. Nothing is assumed about the order of the data.
- **Preprocessing:** None.
- **Why chosen:** It is the only method that works directly on unsorted data and it is the yardstick against which the other two methods must justify their extra preparation.

```
LinearSearch(A[0..n-1], key)
  i <- 0
  while i < n do
      if A[i] = key then return i
      i <- i + 1
  return -1
```

Approach B: Binary search, recursive

- **Design strategy:** Decrease-and-conquer (decrease by a factor of 2).
- **Idea:** In a sorted array, look at the middle record. If it is the key we are done; otherwise the key can only lie in one half, so the other half is thrown away and the same step is repeated on what is left.
- **Preprocessing:** Sort the records by Registration Number with a merge-based sort, Θ(n log n). (The implementation uses Python's built-in Timsort, which is merge-based with the same bound.)
- **Why chosen:** It shows how much ordered data helps, and it is naturally recursive, which is what Tasks 6 and 7 require.

```
BinSearch(A, key, low, high)
  if low > high then return -1                // base case: empty range
  m <- low + floor((high - low) / 2)
  if A[m] = key then return m
  if key < A[m] then return BinSearch(A, key, low, m - 1)
  return BinSearch(A, key, m + 1, high)
```

Approach C: Hash table with separate chaining

- **Design strategy:** Space-time trade-off (transform and conquer): the key is converted into a table index, so no scanning of the whole data is needed.
- **Idea:** Compute the slot h(key) = key mod m. Only the short list (chain) stored in that slot has to be examined.
- **Preprocessing:** Allocate m ≈ 1.5n slots (an odd number) and insert all n records. This costs Θ(n) time and Θ(n) extra space, giving a load factor α = n/m ≈ 0.67.
- **Why chosen:** A lookup by exact Registration Number is exactly the job a hash table is built for, and it is the approach with the best possible expected cost.

```
BuildTable(A, m)
  create T[0..m-1], every slot an empty list
  for each record x in A do
      append x to the list T[x mod m]

FindInTable(T, m, key)
  chain <- T[key mod m]
  for each x in chain do
      if x = key then return found
  return not found
```

Why these are three genuinely different strategies: A inspects records one by one, B repeatedly eliminates half of an ordered range, and C computes where the record must be instead of searching for it. The iterative requirement is met by A and the recursive requirement by B.

TASK 2: BASIC OPERATION

Algorithm	Basic operation	Where it occurs	Why it is appropriate	How its count changes with n
Linear	Key comparison A[i] = key	Test inside the while loop	Executed once per iteration; the index increment and loop test are proportional to it, so it fixes the running time	Grows in direct proportion to n
Binary	Key comparison with A[m] (equality test, then a less-than test)	Once per recursive call	Every call does exactly one such comparison, and every call halves the range, so the number of calls is the number of comparisons	Increases by about 1 each time n doubles
Hash	Key comparison against records in the chain	Chain-traversal loop in FindInTable	The single mod computation is a fixed cost per search; the only part that varies is how many chain entries are compared	Stays almost constant, because the load factor α = n/m is held at about 0.67

In each algorithm the comparison is the step whose repetition is controlled by the input size, which is why counting it predicts the running time. For linear search it is repeated across the data, for binary search it is repeated once per halving, and for hashing it is repeated only along one short chain.

TASK 3: TIME-COMPLEXITY FUNCTIONS

Input size: n = number of records in the database. q = number of searches performed.

Linear search

- **Worst case** (key in the last position or absent): $C(n) = \Sigma_{i=0}^{n-1} 1 = \mathbf{n}$.
- **Best case** (key first): C = 1.
- **Average case**, each position equally likely:
  - Successful: $C = (1 + 2 + \ldots + n)/n = (n + 1)/2$.
  - Unsuccessful: C = n.
  - Mixture with a fraction p of successful queries: $C_{avg} = p(n + 1)/2 + (1 - p)n$. For p = 1/2 this is 0.75n + 0.25.
- **Preprocessing:** 0. **Total for q searches:** at most q·n comparisons.

Binary search

After j comparisons at most $\lfloor n/2^j \rfloor$ records are still in the range. The search is over when the range is empty, i.e. $\lfloor n/2^j \rfloor = 0$, which first happens when $2^j > n$, so $j = \lfloor \log_2 n \rfloor + 1$.

- **Worst case:** $C(n) = \lfloor \log_2 n \rfloor + 1$. For $n = 10^6$ this is $\lfloor 19.93 \rfloor + 1 = 20$.
- **Best case:** 1 (key at the first midpoint).

- **Average case:** Take a perfectly balanced decision tree with $n = 2^h - 1$ nodes. Level i has 2^{i-1} nodes, each needing i comparisons, so for a successful search $C_{avg} = (1/n)\cdot\Sigma_{i=1}^{h} i\cdot 2^{i-1} = ((h-1)2^h + 1)/n = h - 1 + h/n \approx \log_2(n+1) - 1$. An unsuccessful search ends at one of the $n + 1 = 2^h$ gaps, all at depth h, so it costs $h \approx \log_2(n+1)$. With 50% successful queries the mean is about $\log_2(n+1) - 0.5$, i.e. 19.43 for $n = 10^6$.
- **Preprocessing:** Merge-based sort. Its recurrence $S(n) = 2S(n/2) + n$ gives $\Theta(n \log n)$; for $n = 10^6$ that is about $n\cdot\log_2 n \approx 19.9 \times 10^6$ comparisons.
- **Total for q searches:** $n \log n + q \log n$.

Hash table (separate chaining)

Let $\alpha = n/m$ be the load factor; here $m \approx 1.5n$ so $\alpha \approx 0.667$. Assuming simple uniform hashing, a chain has α entries on average.

- Unsuccessful search: expected comparisons = $\alpha \approx 0.67$.
- Successful search: expected comparisons $\approx 1 + \alpha/2 \approx 1.33$.
- For 50% hits and 50% misses: $0.5(1 + \alpha/2) + 0.5\alpha = 0.5 + 0.75\alpha \approx$ **1.00**.
- Each search also performs one mod operation, so $T_{search} = \Theta(1 + \alpha)$, a constant because α is fixed.
- **Worst case:** every key lands in one slot, giving a single chain of n records and n comparisons. This needs a pathological key pattern, which random Registration Numbers do not produce.
- **Preprocessing:** allocating $m = \Theta(n)$ slots plus n insertions of O(1) each gives $\Theta(n)$.
- **Total for q searches:** $n + q(1 + \alpha) = \Theta(n + q)$.

Combined cost (preprocessing + q searches)

Approach	Preprocessing	Per search	Total for q searches
Linear	0	$\Theta(n)$	$\Theta(q\cdot n)$
Binary	$\Theta(n \log n)$	$\Theta(\log n)$	$\Theta((n + q) \log n)$
Hash	$\Theta(n)$	$\Theta(1)$ expected	$\Theta(n + q)$

Break-even, binary vs linear (counting comparisons): sorting pays for itself when $n \log n + q \log n < 0.75\cdot q\cdot n$. Dividing by n, this is $q > \log n / 0.75$ approximately, which for $n = 10^6$ means q is about 27 searches or more.

TASK 4: BEST, AVERAGE AND WORST CASES

Linear search

- **Best:** the key is the very first record, so one comparison settles it.
- **Average:** if the key is present and equally likely to be anywhere, about $(n + 1)/2$ records are examined. A missing key always costs n, so as more queries fail the average moves from n/2 towards n.
- **Worst:** the key is the last record or not in the database. Nothing can be excluded without being read, so all n records are compared.

Binary search

- **Best:** the key happens to be the middle record of the full range: one comparison.
- **Average:** in the decision tree half of all nodes sit on the bottom level and a quarter on the one above, so a randomly chosen existing key usually needs close to the maximum depth. This is why the average is only about one comparison below the worst: $\log_2(n+1) - 1$.
- **Worst:** the key is absent, or sits on the deepest level. The range can be halved at most $\lfloor \log_2 n \rfloor + 1$ times before it becomes empty.

Hash table

- **Best:** the key is the first entry in its chain (or the chain is empty for an absent key, giving zero comparisons).
- **Average:** chains are $\alpha \approx 0.67$ long on average, so a hit costs $1 + \alpha/2$ and a miss costs α. Both are constants independent of n.

- **Worst:** all keys share one slot, producing one chain of length n. It requires a very poor hash function or deliberately chosen keys; in the experiment the longest chain at $n = 10^6$ was only 7 entries.

TASK 5: ASYMPTOTIC ANALYSIS

Algorithm	Best	Average	Worst	Notation for the running time
Linear	Ω(1)	Θ(n)	Θ(n)	T(n) = O(n) and Ω(1); worst/average cases are Θ(n)
Binary (search only)	Ω(1)	Θ(log n)	Θ(log n)	T(n) = O(log n)
Binary (sort + q searches)				Θ(n log n + q log n)
Hash (search only)	Ω(1)	Θ(1)	O(n)	Expected Θ(1), worst case O(n)
Hash (build + q searches)				Θ(n + q) expected

From 1,000 to 1,000,000 records (a factor of 1000):

- Linear search does about 985 times more work (751 → 739,919 comparisons). A lookup that feels instant on a small table becomes the bottleneck on a large one.
- Binary search goes from about 9.5 to 19.5 comparisons, only about 2.1 times more, although the data grew 1000-fold.
- Hash search stays at about one comparison (1.01 → 1.01); its cost does not depend on n.

For the university this means linear search is only acceptable for small or one-off lookups, while binary and hash search both remain practical at 1,000,000 records and well beyond. The difference between them is that binary search still grows (slowly) with the database, whereas hashing does not.

TASK 6: RECURRENCE RELATION (RECURSIVE BINARY SEARCH)

Component	Value for BinSearch
Input size of original problem	n = high − low + 1 records in the current range
Size of subproblem	At most $\lfloor n/2 \rfloor$ (the left part or the right part, never both)
Number of recursive calls	1 (at most one half is searched)
Work outside the call	Computing m and one comparison with A[m]: constant, Θ(1)
Base case	Empty range (n = 0) returns at once, T(0) = 0; one record needs one comparison, T(1) = 1

Derivation: each call spends constant time on m and the comparison, then makes a single call on a range of at most half the size. Writing the constant as 1 comparison gives

$$\mathbf{T(n) = T(\lfloor n/2 \rfloor) + 1 \text{ for } n \geq 2, T(1) = 1.}$$

TASK 7: SOLVING THE RECURRENCE

Method 1: Iteration (backward substitution). The recurrence has one recursive call and a constant extra term, so repeated expansion shows a clear pattern. Take $n = 2^k$ to remove the floors:

```
T(n) = T(n/2) + 1
     = T(n/4) + 1 + 1
     = T(n/8) + 3
     ...
     = T(n/2^j) + j
```

The expansion stops when $n/2^j = 1$, i.e. $j = \log_2 n$. So $T(n) = T(1) + \log_2 n = 1 + \log_2 n =$ **$\Theta(\log n)$**.

Method 2: Master Theorem. The form is $T(n) = aT(n/b) + f(n)$ with $a = 1$, $b = 2$, $f(n) = 1$. Then $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^{\log_b a})$, Case 2 applies: $T(n) = \Theta(n^0 \log n) =$ **$\Theta(\log n)$**.

Method 3: Substitution (induction). Claim: $T(n) \leq \log_2 n + 1$ for all $n \geq 1$.

- Base: $T(1) = 1 \leq \log_2 1 + 1 = 1$ ✓.

- Step: assume the claim for all smaller values. For $n \geq 2$, $T(n) = T(\lfloor n/2 \rfloor) + 1 \leq \log_2(n/2) + 1 + 1 = \log_2 n - 1 + 2 = \log_2 n + 1$ ✓.

Interpretation: the cost is logarithmic. Every doubling of the database adds a single comparison. For $n = 10^6$ the bound gives at most 20.9, so 20 comparisons, and one billion records would need about 30. The recursion depth is also $\Theta(\log n)$, so the stack use is tiny, which keeps recursion practical here.

TASK 8: IMPLEMENTATION AND TESTING

The three algorithms were implemented in Python 3 exactly as in Task 1. The full program that produced all results in this report is listed below; it counts comparisons and measures time.

```
import random, time, sys, json
sys.setrecursionlimit(10000)

def linear_search(a, k):
    c = 0
    for i in range(len(a)):
        c += 1
        if a[i] == k:
            return i, c
    return -1, c

def binary_search(a, k, lo, hi, cnt):
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    cnt[0] += 1
    if a[mid] == k:
        return mid
    if k < a[mid]:
        return binary_search(a, k, lo, mid - 1, cnt)
    return binary_search(a, k, mid + 1, hi, cnt)

def build_table(a, m):
    t = [[] for _ in range(m)]
    for x in a:
        t[x % m].append(x)
    return t

def hash_search(t, m, k):
    c = 0
    for x in t[k % m]:
        c += 1
        if x == k:
            return True, c
    return False, c

# ---- edge-case tests ----
def edge_tests():
    data = sorted(random.sample(range(10**8, 10**9), 101))
    m = int(len(data) * 1.5) | 1
    t = build_table(data, m)
    cases = {
        "first": data[0], "last": data[-1], "middle": data[50],
        "smaller than all": 5, "larger than all": 10**10,
    }
    for name, k in cases.items():
        exp = k in data
        assert (linear_search(data, k)[0] != -1) == exp, name
        assert (binary_search(data, k, 0, len(data) - 1, [0]) != -1) == exp, name
        assert hash_search(t, m, k)[0] == exp, name
    one = [123456789]
    t1 = build_table(one, 3)
```

```
    for k, exp in [(123456789, True), (987654321, False)]:
        assert (linear_search(one, k)[0] != -1) == exp
        assert (binary_search(one, k, 0, 0, [0]) != -1) == exp
        assert hash_search(t1, 3, k)[0] == exp
    return "all edge cases passed"

random.seed(2024)
print(edge_tests(), flush=True)

results = []
for n in [1000, 10000, 100000, 1000000]:
    random.seed(2024 + n)
    keys = random.sample(range(10**8, 10**9), n)
    present = set(keys)
    nq = 400 if n <= 10000 else 100
    hits = random.sample(keys, nq // 2)
    misses = []
    while len(misses) < nq // 2:
        x = random.randrange(10**8, 10**9)
        if x not in present:
            misses.append(x)
    qs = hits + misses
    random.shuffle(qs)

    # linear
    t0 = time.perf_counter(); tc = 0
    for q in qs:
        r, c = linear_search(keys, q); tc += c
        assert (r != -1) == (q in present)
    lin_t = (time.perf_counter() - t0) / nq * 1000
    lin_c = tc / nq

    # binary
    t0 = time.perf_counter(); sk = sorted(keys)
    sort_t = (time.perf_counter() - t0) * 1000
    cnt = [0]; t0 = time.perf_counter()
    for q in qs:
        r = binary_search(sk, q, 0, n - 1, cnt)
        assert (r != -1) == (q in present)
    bin_t = (time.perf_counter() - t0) / nq * 1000
    bin_c = cnt[0] / nq

    # hash
    m = int(n * 1.5) | 1
    t0 = time.perf_counter(); ht = build_table(keys, m)
    hb_t = (time.perf_counter() - t0) * 1000
    tc = 0; t0 = time.perf_counter()
    for q in qs:
        r, c = hash_search(ht, m, q); tc += c
        assert r == (q in present)
    h_t = (time.perf_counter() - t0) / nq * 1000
    h_c = tc / nq
```

```
    longest = max(len(b) for b in ht)

    row = dict(n=n, nq=nq, lin_c=lin_c, bin_c=bin_c, h_c=h_c,
               lin_t=lin_t, bin_t=bin_t, h_t=h_t,
               sort_t=sort_t, hb_t=hb_t, m=m, alpha=n / m, longest=longest)
    results.append(row)
    print(row, flush=True)

json.dump(results, open("results.json", "w"), indent=1)
```

Test procedure

- For each size n ∈ {1,000; 10,000; 100,000; 1,000,000} the same set of n unique 9-digit Registration Numbers (fixed random seed) is given to all three algorithms.
- The same query list is used for all three: 400 queries for n ≤ 10,000 and 100 for the larger sizes (linear search is too slow to repeat more at 1,000,000 records). Half the queries are existing keys and half are

keys that are not in the database, in shuffled order.

- Every answer from every algorithm is checked with assert against Python's set membership, so wrong results would stop the run. All runs passed.
- Edge-case tests (also passed): first key, last key, middle key, key smaller than every record, key larger than every record, and a database with a single record (key present and absent).
- Search time is the mean per query; sorting and hash-table construction are timed separately, so preprocessing can be added in the analysis exactly as the assignment requires.

TASK 9: EXPERIMENTAL PERFORMANCE

Measured on one machine with Python 3, using identical data for all algorithms. Times depend on hardware, so absolute values will differ on another machine; the growth pattern is what matters.

Average comparisons per search (50% hits, 50% misses)

n	Linear	Binary	Hash
1,000	750.98	9.49	1.01
10,000	7,606.32	12.82	0.97
100,000	71,787.89	16.27	0.95
1,000,000	739,919.17	19.46	1.01

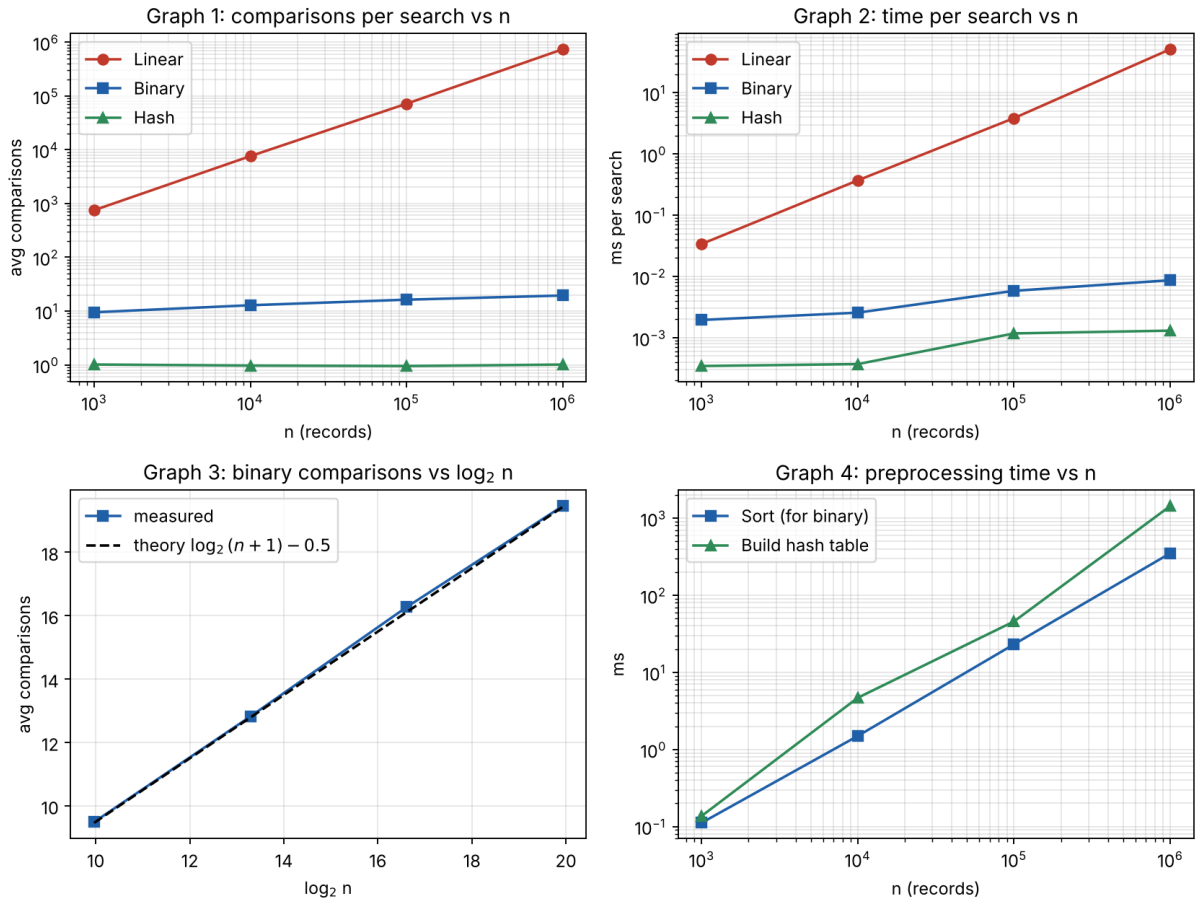
Average time per search (ms)

n	Linear	Binary	Hash
1,000	0.0339	0.0019	0.0003
10,000	0.3708	0.0026	0.0004
100,000	3.8517	0.0058	0.0012
1,000,000	51.3174	0.0087	0.0013

Preprocessing time (ms) and hash-table statistics

n	Sort (for binary)	Build hash table	Slots m	Longest chain
1,000	0.11	0.14	1,501	4
10,000	1.49	4.68	15,001	5
100,000	23.10	45.73	150,001	7
1,000,000	350.21	1,435.44	1,500,001	7

The sort uses Python's built-in routine, which is implemented in optimised C, whereas the hash table is built by a loop written in Python. The two preprocessing times are therefore not directly comparable in absolute terms, although both grow with n as expected.



TASK 10: THEORY VS EXPERIMENT

Linear search. Each tenfold increase in n multiplies the comparisons by about 10: 751 → 7,606 → 71,788 → 739,919. Dividing by n gives 0.751, 0.761, 0.718 and 0.740, close to the predicted 0.75n for a 50/50 mixture. This confirms Θ(n); the small scatter is sampling variation from the limited number of queries.
Binary search. The count rises by 3.33, 3.45 and 3.19 per tenfold increase in n, in line with $\log_2 10 \approx 3.32$. At $n = 10^6$ the measured 19.46 compares with the predicted mean of 19.43 and the worst case of 20. This confirms Θ(log n).
Hash table. The comparison count stays between 0.95 and 1.01 at every size, matching the predicted 1.00 for $\alpha \approx 0.67$. This confirms Θ(1) expected.
Which scales best: hash first, binary second, linear last.

Where theory and measurement differ, and why

- Time grows faster than the comparison count for binary and hash search. Binary time rose 4.5× although comparisons rose only 2.1×; hash time rose 3.8× while its comparisons stayed flat. At 1,000,000 records the data no longer fits in the CPU cache, so each memory access costs more, and a probe into a large table or array is effectively a random access.
- Even linear search shows this: its time per comparison grew from about 45 ns at n = 1,000 to about 69 ns at $n = 10^6$, because the Python list holds pointers to separate integer objects.
- Recursion adds function-call overhead per level in Python, which the comparison count does not capture.
- Chain lengths vary from slot to slot, and timings of a few microseconds are affected by interpreter and operating-system noise.
- Only 100 queries were used at the two largest sizes, so those averages carry more sampling error.

TASK 11: FINAL DESIGN RECOMMENDATION

Recommendation for the original problem: a sorted array searched with binary search.

- **Why it beats the alternatives:** Linear search is 38,023 times more expensive in comparisons at 10^6 records. Compared with hashing, the sorted array needs no extra table (about 50% more slots), gives a guaranteed worst case instead of a probabilistic one, and supports range and ordered queries, for example all students with Registration Numbers between two values, or a sorted listing.
- **Theoretical efficiency:** $\Theta(\log n)$ per search with $\Theta(n \log n)$ preprocessing.
- **Observed performance:** 19.5 comparisons and 0.0087 ms per search at 10^6 records, against 739,919 comparisons and 51.3 ms for linear search.
- **Preprocessing:** sorting 10^6 records took about 0.35 s. Measured by time it repays itself after roughly 7 searches; counted in comparisons the break-even is about 27 searches.
- **Trade-offs and limitations:** inserting a single record into a sorted array costs O(n) because later records must shift, so it suits data that is read far more often than it is written. For a tiny table or a one-off lookup, plain linear search is simpler and sorting would not pay off.
- **Scalability:** because the cost is logarithmic, a database of one billion records would still need only about 30 comparisons per search.

TASK 12: DAY-4 DESIGN CHALLENGE

New conditions: 50,000 searches per hour on 10^6 records, with new records added once per day.

A. Impact Analysis

- **What changed:** the question is no longer a single lookup on a growing table but a sustained stream of repeated lookups on an almost static table: 50,000/3600 ≈ 13.9 searches per second, or 1.2 million per day.
- **Assumptions affected:** the original analysis weighed a one-time preprocessing cost against a small number of queries. Now q is huge, so the per-search cost dominates the total and the one-off preprocessing becomes minor.
- **Why search frequency matters:** any saving per search is multiplied by about 1.2 million every day.
- **Why rare updates matter:** sorting or building a hash table is needed only once per day and can be done overnight in a batch. The usual weaknesses of both structures (O(n) insertion in a sorted array, rehashing for a hash table) are therefore almost irrelevant.

B. Evaluation of the Original Design

- **Behaviour under load:** binary search needs about 19.5 comparisons per query, so 50,000 queries are about 973,000 comparisons per hour. Measured at 0.0087 ms each, that is only about 0.43 s of CPU time per hour. The original design copes easily.
- **Preprocessing:** the 0.35 s sort is done once a day and is negligible next to 1.2 million searches.
- **Could another approach do better?** Yes. Hashing needs about 1.01 comparisons per search, i.e. 50,500 per hour, around 19 times fewer than binary search. By measured time it is 6.7× faster (0.0013 ms vs 0.0087 ms).
- **Linear search for comparison:** $50{,}000 \times 739{,}919 \approx 3.7 \times 10^{10}$ comparisons per hour and 2,566 s of CPU per hour, i.e. about 71% of one core running non-stop. It is clearly unsuitable.

C. Adaptation: MODIFY the design

Revised strategy: keep the sorted array and binary search, but put a hash table in front as the primary index for exact-match lookups.

- **What changes:** exact Registration Number queries go to the hash table (expected $\Theta(1)$). The sorted array is kept for range queries, ordered listings and as an independent check of the table.
- **Daily update:** new records are collected during the day and applied in one nightly batch. The k new records are sorted ($O(k \log k)$) and merged into the sorted array ($O(n + k)$); the hash table is rebuilt in $\Theta(n)$ in a separate copy, and then swapped in, so searches never see a half-built table.
- **Effect on complexity:** per search $O(\log n) \rightarrow \Theta(1)$ expected. For $q = 1.2 \times 10^6$ daily searches the work falls from about $q \cdot \log_2 n \approx 23.4 \times 10^6$ comparisons to about 1.21×10^6. Preprocessing is $\Theta(n \log n) + \Theta(n)$ per day, and extra memory is about 1.5 million table slots.

D. Evidence-Based Defence

- **Search frequency:** 13.9 searches per second and 1.2 million per day make the per-search cost the dominant term.
- **Complexity:** total work is $q \cdot n$ for linear, $n \log n + q \log n$ for binary, and $n + q$ for hash. With $q = 1.2 \times 10^6$ and $n = 10^6$, hash is lowest.
- **Experiment:** at 10^6 records hash used 1.01 comparisons per search against 19.46 for binary and 739,919 for linear; per-search time was 6.7× lower than binary and about 39,375× lower than linear.
- **Preprocessing:** the hash build took 1.44 s (pure Python) against 0.35 s for the sort. The extra 1.09 s is recovered after about 147,297 searches, which is under 3 hours of traffic at 50,000 per hour. Over a day hash saves about 8.8 s of CPU.
- **Honest assessment:** at today's load even binary search alone would use well under a second of CPU per hour (10 s per day), so the change is not forced by capacity. It is justified because hashing is cheaper still, its cost does not grow with n, and it leaves headroom if the load or the database grows, while the retained sorted array preserves ordered and range queries.
- **Trade-offs:** the table needs about 50% more slots than records, has no ordering, and has an O(n) worst case, which a sound hash function and random-like Registration Numbers make very unlikely.

Final decision: MODIFY. Use a hash table for exact-match lookups, keep the sorted array with binary search for range and ordered queries, and refresh both in a nightly batch.